

05-18 Лекция: Автоматизация с помощью ИИ-агентов, серверные и локальные решения

[Дата и время] : 2026-05-18 13:14:46

[Местоположение: [Вставить местоположение]]

[Инструктор: [Вставить имя]], Николай

Резюме

В лекциях обсуждаются различные подходы и технические аспекты автоматизации задач с помощью ИИ-агентов. Рассматриваются как серверные, так и локальные решения, сравниваются их надежность и сферы применения. Ключевое внимание уделяется серверным технологиям, таким как `cron` для запуска задач по расписанию, `systemd` для обеспечения непрерывной работы приложений (демонов) и `webhook` для реакции на внешние события в реальном времени. Обсуждается архитектура сложных систем на основе очередей задач (с использованием `Redis` и `Celery`), необходимых для управления взаимосвязанными и ресурсоемкими процессами.

На практических примерах демонстрируется создание, развертывание и усложнение Telegram-бота на Python, который интегрируется с языковыми моделями (через `OpenRouter`) и запускается как постоянная служба на VPS с помощью `systemd`. В заключение проводится сравнение двух подходов к созданию автоматизаций: гибкого написания кода и использования визуальных no-code/low-code платформ типа `n8n`. Подчеркивается, что прямой кодинг обеспечивает больший контроль и масштабируемость для сложных систем, в то время как no-code платформы удобны для простых задач и визуализации архитектуры, но быстро усложняются при росте проекта.

Основные положения

1. Механизмы автоматизации и их типы

- **Heartbeat (Пульт):** Серверный механизм, при котором основной агент периодически “просыпается”, анализирует свой полный контекст и выполняет сложные задачи. Этот метод требует значительного количества токенов.
- **Cron (Крон):** Стандартная утилита в Linux для запуска задач по фиксированному расписанию (например, ежедневно в 9 утра). Идеально подходит для простых, узкоспециализированных задач, выполняемых субагентами, таких как отправка сводки новостей или выставление счетов. В отличие от `systemd`, `cron` запускает скрипт, дожидается его выполнения и завершает процесс.
- **WebHooks (Вебхуки):** Механизм, при котором внешняя система (API) сама отправляет уведомление (триггер) при наступлении события (например, новое сообщение в Telegram). Это позволяет реагировать на события мгновенно, избегая необходимости постоянно опрашивать API.
- **Очереди задач:** Архитектурный подход для управления сложными, взаимосвязанными или ресурсоемкими рабочими процессами. Система состоит из **очереди** (хранилища задач), **исполнителей** (воркеров, которые берут задачи на выполнение) и **логов** для отслеживания истории. Рекомендуемые инструменты для реализации — `Redis` (в качестве базы данных для очереди) и `Celery` (фреймворк для управления очередями).

2. Сравнение локальных и серверных (облачных) решений

- **Локальные решения (Codex):**
 - Автоматизация работает на компьютере пользователя и функционирует только тогда, когда компьютер включен и приложение запущено.
 - Пропущенные по времени триггеры не выполняются постфактум.
 - Из-за низкой надежности подходит для некритичных задач, но не для бизнес-процессов, требующих гарантированного выполнения по расписанию.
- **Серверные решения (VPS):**
 - Работают непрерывно на виртуальном сервере, независимо от состояния компьютера пользователя.
 - Обеспечивают высокую надежность и постоянную доступность.
 - Для обеспечения постоянной работы приложений используется `systemd` — служба в Linux, которая автоматически запускает программы при старте сервера и перезапускает их в случае сбоя.
 - Для повышения отказоустойчивости в профессиональных системах используется несколько VPS, которые могут контролировать

работоспособность друг друга.

3. Практическое применение: создание и развертывание Telegram-бота

- **Создание бота:** Демонстрируется создание бота на Python, начиная от простого эхо-бота до “умного” ассистента, интегрированного с языковой моделью (Google Gemini 3.1 через OpenRouter).
- **Конфигурация:** Боту задается роль (системный промпт), например, психолога по КПТ, и добавляется “память” для хранения контекста диалога.
- **Развертывание (деплой):** Для обеспечения постоянной работы бот развертывается на VPS. Его скрипт запускается как служба с помощью systemd . Это гарантирует, что бот будет работать 24/7 и автоматически перезапустится после перезагрузки сервера или сбоя.
- **Безопасность:** API-ключи и другие чувствительные данные хранятся на сервере в файлах окружения (.env) и не попадают в публичные репозитории.
- **Важность логирования:** При проектировании фоновых задач (cron, воркеры) крайне важно вести лог-файлы для отслеживания выполнения, отладки и контроля ошибок.

4. Сравнение подходов: Код vs. No-code (N8N)

- **Написание кода (Python):**
 - **Преимущества:** Неограниченная гибкость (“3D-печать”), полный контроль над логикой, возможность подключения любых систем, простота отладки сложных систем по логам, лучшая масштабируемость.
 - **Недостатки:** Требует навыков программирования.
 - **Стоимость:** Оплата за использование токенов языковой модели.
- **No-code/Low-code платформы (N8N):**
 - **Преимущества:** Визуальная наглядность (“конструктор LEGO”), низкий порог входа для простых задач, удобно для демонстрации архитектуры.
 - **Недостатки:** С ростом сложности логика на экране становится громоздкой и неуправляемой. Отладка в сложных визуальных схемах затруднена. Ограниченные возможности кастомизации.
 - **Вывод:** N8N и подобные платформы хороши для простых автоматизаций и прототипирования, но для сложных и масштабируемых систем кодовый подход предпочтительнее.

Вопросы

- [Вставить вопрос/неясность]

Задания

- [] 1. Изучить разницу между механизмами автоматизации `Heartbeat` , `cron` , `systemd` и `Webhook` .
- [] 2. Разобраться с работой VPS и настроить на нем автоматизацию с использованием `cron` для периодического запуска скрипта (например, отправляющего сообщение в Telegram).
- [] 3. При проектировании сложных систем рассмотреть использование очередей задач с помощью `Redis` и `Celery` .
- [] 4. Создать собственного Telegram-бота на Python, интегрировать его с языковой моделью через `OpenRouter` .
- [] 5. Развернуть (задеплоить) созданного бота на VPS, используя `systemd` для обеспечения его постоянной работы, и продумать архитектуру хранения данных.
- [] 6. Зарегистрироваться на платформе `N8N` и попробовать создать простые рабочие процессы (workflow), чтобы сравнить подход с прямым кодированием.